

```

        wrmsrl(MSR_LSTAR, (unsigned long)entry_
SYSCALL_64);
.
.
}

/*Now let's see what in detail about entry_SYSCALL_64 */
arch/x86/entry/entry_64.S

ENTRY(entry_SYSCALL_64)
.
.

/* This SWAPGS_UNSAFE_STACK macro uses
 * swapgs instruction switches to kernel stack
 * for switching the stack swapgs instruction uses segment
 * register GS and MSR_KERNEL_GS_BASE
 */

        SWAPGS_UNSAFE_STACK

/* Move the old stack pointer to per cpu variable
 * and setup the stack pointer to point
 * to the top of stack for the current processor:
 */

        GLOBAL(entry_SYSCALL_64_after_swapgs)

        movq %rsp, PER_CPU_VAR(rsp_scratch)
        movq PER_CPU_VAR(cpu_current_top_of_stack), %rsp

/* Construct struct pt_regs on stack
 * We are creating stack frame for system call handler
 * we are actually moving user stack segment,
 * stack pointer and other general-purpose register
 * on to the kernel stack. Remember general purpose register
 * state when system call happens from user space.
 * we push register flags on the top of the stack,
 * then user code segment, return address
 * to the user space, system call number, and system call
arguments
 */

TRACE_IRQS_OFF

/* Construct struct pt_regs on stack */
pushq$_USER_DS          /* pt_regs->ss */
pushqPER_CPU_VAR(rsp_scratch) /* pt_regs->sp */
pushq%r11               /* pt_regs->flags */
pushq$_USER_CS          /* pt_regs->cs */
pushq%rcx               /* pt_regs->ip */
pushq%rax               /* pt_regs->orig_ax */
pushq%rdi               /* pt_regs->di */

```

```

pushq%rsi               /* pt_regs->si */
pushq%rdx               /* pt_regs->dx */
pushq%rcx               /* pt_regs->cx */
pushq$-ENOSYS           /* pt_regs->ax */
pushq%r8                /* pt_regs->r8 */
pushq%r9                /* pt_regs->r9 */
pushq%r10               /* pt_regs->r10 */
pushq%r11               /* pt_regs->r11 */
sub $(6*8), %rsp        /* pt_regs->bp, bx, r12-15 not
saved */

...
...
...
entry_SYSCALL_64_fastpath:

/*
 * Easy case: enable interrupts and issue the syscall. If
the syscall
 * needs pt_regs, we'll call a stub that disables interrupts
again
 * and jumps to the slow path.
 */

        TRACE_IRQS_ON
        ENABLE_INTERRUPTS(CLBR_NONE)

/* Here basically we are comparing the system call number
(eax)
 * with NR_syscall_max. Remember NR_syscall_max
 * and sys_call_table that we had discussed
 */

#if __SYSCALL_MASK == ~0
        cmpq $__NR_syscall_max, %rax
#else
        andl $__SYSCALL_MASK, %eax
        cmpl $__NR_syscall_max, %eax
#endif
        jaif /* return -ENOSYS (already in pt_
regs->ax) */

/* We move the fourth argument from the r10 to the rcx
 * This move of r10 to rcx is as per the register calling
 * convention difference for passing the system call
 * argument through syscall instruction (we had discussed
this earlier)
 * Now execute the call instruction with the address of
system call handler
 * rax contain system call number and each array element of
 * sys_call_table contain pointer to system call handler
 * This pointer to system call handler is of 8 byte
 * hence *sys_call_table(, %rax, 8) find the correct offset

```